



Università
di Catania

Dipartimento di
Ingegneria Elettrica Elettronica e
Informatica

Advanced Programming Languages

C++: special members and RAII

Docente:
Marco Grassia
marco.grassia@unict.it

A.A. 2025/2026

Special member functions

Advanced Programming Languages: C++

C++ classes: the six “special” member functions

Some of the member functions that are fundamental for object lifecycle (creation, destruction, copying, moving) **are called “special”:**

- 1. Default constructor**
- 2. Destructor**
- 3. Copy constructor**
- 4. Copy assignment operator**
- 5. Move constructor (C++11+)**
- 6. Move assignment operator (C++11+)**

C++ classes: why “special” member functions and what’s their purpose?

- Essential for **RAII** and modern C++ design: they control **ownership semantics** (copy vs move), **exception guarantees**, and **triviality** (important for ABI and optimization)
- Misusing them can lead to **double deletes**, **resource leaks**, or **unexpected copies**
- The compiler can **implicitly declare and define** them for you
- Their **presence** or **absence affects class behavior**, **performance**, and **safety**
- **Declaring** or omitting one can **suppress others** (e.g., custom destructor disables implicit move)

C++ classes: special member functions

- The **compiler *may* implicitly generate them based** on what your class does
- **Declaring some members can suppress others** (e.g., user-declared destructor suppresses implicit move)
- **You can force or forbid** generation:
 - = **default** requests the compiler-provided implementation
 - = **delete** forbids use (compile-time error)

C++ classes: special member functions

= default

- = **default** explicitly **asks** the **compiler to generate the default implementation** of a special member function
 - **Useful for clarity and performance:** the compiler-generated version is often more efficient than a hand-written empty body
 - **It does not change how members are initialized:** uninitialized fundamental members remain indeterminate unless you provide initializers

```
struct Plain {  
    // indeterminate if default-initialized  
    int x;  
    // does not zero-init x  
    Plain() = default;  
};  
  
struct Zeroed {  
    // member initializer  
    int x = 0;  
    // x is 0  
    Zeroed() = default;  
};
```

C++ classes: special member functions

= default –ed copy and move

- **Defaulted copy** performs **member-wise copy**:
 - Fundamental types → bitwise copy
 - Class-type members → call their copy constructor/assignment
- **Defaulted move** performs **member-wise move**:
 - Fundamental types → bitwise copy
 - Class-type members → call their move if available, else copy
- **Caution: pointers have no move semantics:** moving a raw pointer just copies the address. Copying raw owning pointers → shallow copy → double delete or leaks

```
struct BadOwner {  
    int* p;  
    // shallow copy!  
    BadOwner(const BadOwner&) = default;  
  
    // Move constructor just copies pointer  
    BadOwner(BadOwner&&) = default;  
  
    // If a copy is made, both objects  
    // will try to delete the same pointer  
    // leading to double deletion and  
    // undefined behavior.  
    ~BadOwner() { delete p; }  
};
```

C++ classes: special member functions

= delete

- = `delete` explicitly **forbids calling/instantiating a function** (including special members) **producing a compile-time error**
- The function **still participates in overload resolution**: it's a **stronger message than just declaring it private**
- **Useful to:**
 - **Disable copy/move for unique ownership**
 - **Reject undesired conversions**

```
struct NonCopyable {  
    NonCopyable() = default;  
    NonCopyable(const NonCopyable&) = delete;  
    NonCopyable& operator=  
        (const NonCopyable&) = delete;  
  
    // Cannot be converted to std::string  
    operator std::string() const = delete;  
};
```


Resource Acquisition Is Initialization (RAII)

Advanced Programming Languages: C++

C++ manual resource management

- Up to now, **we've** how **to manage** resources **manually** with new and delete
- However, **this is error prone**: memory leaks, double delete, dangling pointers can happen easily
- **Exception safety is hard**: if an exception occurs before delete, resource leaks
- If there are multiple resources, the cleanup logic explodes. E.g., only cleanup the resources that have been already allocated

```
void process(std::size_t n) {  
    // manual allocation  
    char* buf = new char[n];  
  
    // some work...  
  
    // exception thrown  
    throw std::runtime_error("boom");  
  
    // never reached → memory leak  
    delete[] buf;  
}
```

Resource Acquisition Is Initialization (RAII)

- C++'s philosophy: the **object that owns a resource** (memory, file handle, socket, etc.) **is responsible for acquiring it in its constructor and releasing it in its destructor**
- This ensures automatic cleanup **when the object goes out of scope, even in case of exceptions**
- This is the **foundation for safe resource management in C++ without garbage collection**

```
class Buffer {  
    std::size_t size = 0;  
    char* data = nullptr;  
  
public:  
    Buffer() = default;  
    Buffer(std::size_t s) : size(s), data(new char[s]) {}  
  
    ~Buffer() {  
        // RAII: cleanup automatically  
        delete[] data;  
    }  
};
```

```
void process() {  
    Buffer buf(1024); // allocated  
    throw std::runtime_error("error");  
    // destructor runs → cleanup  
}
```

Resource Acquisition Is Initialization (RAII)

- It **avoids manual resource management** (e.g., delete or close) that lead to memory leaks
- It also **guarantees exception safety**: destructors run during stack unwinding when an exception is thrown (as instance goes out of scope)
- Enables **deterministic cleanup** (unlike GC languages)

```
class Buffer {  
    std::size_t size = 0;  
    char* data = nullptr;  
  
public:  
    Buffer() = default;  
    Buffer(std::size_t s) : size(s), data(new char[s]) {}  
  
    ~Buffer() {  
        // RAII: cleanup automatically  
        delete[] data;  
    }  
};
```

```
void process() {  
    Buffer buf(1024); // allocated  
    throw std::runtime_error("error");  
    // destructor runs → cleanup  
}
```

C++ classes: copy semantics and RAI

- If a class owns a resource, **default copy** is shallow:
 - Copies the pointer, not the resource → double delete or leaks
- You must implement **deep copy** or disable copy

```
class Buffer {  
    char* data;  
    std::size_t size;  
public:  
    Buffer(std::size_t s) : size(s), data(new char[s]) {}  
    ~Buffer() { delete[] data; }  
  
    // Copy constructor  
    Buffer(const Buffer& other)  
        : size(other.size), data(new char[size]) {  
        std::copy(other.data, other.data + size, data);  
    }  
};
```

C++ classes: move semantics

- Transfers ownership of a resource from one object to another **without copying** (fast)
- Leaves the source in a **valid but empty state**
- Introduced in **C++11** for efficiency and safety

```
class Buffer {  
    char* data;  
    std::size_t size;  
public:  
    Buffer(std::size_t s) : size(s), data(new char[s]) {}  
    ~Buffer() { delete[] data; }  
  
    // Copy constructor  
    Buffer(const Buffer& other)  
        : size(other.size), data(new char[size]) {  
        std::copy(other.data, other.data + size, data);  
    }  
};
```

C++ classes: move semantics and RAI

- It triggers automatically when **assigning** from **temporaries/prvalues**: e.g.:
T x = T(...); or T x = makeT();
- When **returning a local by value** (copy elision/NRVO in C++17+)
- To move **from a named object** (otherwise you copy), **you need std::move**

```
class Buffer {  
    char* data;  
    std::size_t size;  
public:  
    Buffer(std::size_t s) : size(s), data(new char[s]) {}  
    ~Buffer() { delete[] data; }  
    // Move constructor FROM RVALUE!!  
    Buffer(Buffer&& other) noexcept  
        : data(other.data), size(other.size) {  
        other.data = nullptr;  
        other.size = 0;  
    }  
    // Move assignment FROM RVALUE!!  
    Buffer& operator=(Buffer&& other) noexcept {  
        if (this != &other) {  
            delete[] data;  
            data = other.data;  
            size = other.size;  
            other.data = nullptr;  
            other.size = 0;  
        }  
        return *this;  
    }  
};
```

Smart pointers

Advanced Programming Languages: C++

C++ manual RAII limitations

- **Manual RAII scales poorly** for complex classes
- In fact, **writing custom** destructors, copy/move **logic for every class is tedious and error-prone**
- It requires careful handling of copy/move and exception safety
- For instance, **if an exception occurs in constructor**, the destructor never runs because the object was never fully constructed;

Example: If you allocate multiple resources in a constructor and an exception occurs halfway, cleanup for already-acquired resources must be handled manually

```
class Complex {  
public:  
    Complex() {  
        p1 = new int[100];  
        p2 = new int[200];  
        throw std::runtime_error("boom");  
        // destructor will not be called here  
        // thus, memory leak occurs (p1 and p2 not deleted)  
    }  
    ~Complex() { delete[] p1; delete[] p2; }  
private:  
    int* p1{};  
    int* p2{};  
};
```

C++ smart pointers and Rule of Zero

- Smart pointers are **RAII wrappers for dynamic memory provided by the Standard Library** `<memory>`
- **They automatically manage lifetime and integrate with move semantics**
- They **make ownership explicit** in code (and clear), **reduce boilerplate** (no need to write destructor, copy/move manually) and are **exception-safe by design**

C++ smart pointers: what are they?

- A smart pointer is a **class template** that **behaves like** a pointer but manages ownership automatically
- It expresses ownership intent clearly
- Implement RAII for you: **acquires resource on construction, releases on destruction**
- Exception-safe by design, and **eliminates manual** `delete`
- Supports (and handle correctly) **copy and move semantics** for ownership transfer

C++ smart pointer types:

`unique_ptr<T>`

- `std::unique_ptr<T>` for exclusive ownership (single owner of a resource)
- Non-copyable, movable
- Also supports a custom deleter function

```
{
    std::unique_ptr<int> p1(new int(42));
    // // Error: cannot copy unique_ptr
    // std::unique_ptr<int> p2 = new int(43);

    std::unique_ptr<int[]> arr(new int[10]);
    // Custom deleter:

    std::unique_ptr<int[], void(*)(int*)> arr_custom(
        new int[10],
        [](int p[]) {
            std::cout << "Custom deleting array\n";
            delete[] p;
        }
    );
}
```

C++ smart pointer types:

shared_ptr<T>

- std::shared_ptr<T> implements shared ownership via reference counting
- Affected by cyclic references

```
{  
  
    std::shared_ptr<int> sp1(new int(100));  
    {  
        // Reference count increases  
        std::shared_ptr<int> sp2 = sp1; // shared ownership  
        // Reference count decreases when sp2 goes out of scope  
    }  
    // sp1 still valid here  
  
} // sp1 goes out of scope, memory freed
```

C++ smart pointer helpers: make_unique and make_shared (C++14)

- They are **factory functions** in the C++ Standard Library (header <memory>)
- Their purpose is to **create and return smart pointers** without explicitly using new
- Added for: **safety** (exception-proof allocation); **clarity** (express ownership intent); **efficiency** (especially for make_shared)

```
// unique ownership
```

```
// verbose, risk of leaks
```

```
auto p_old_way = std::unique_ptr<int>(new int(42));
```

```
auto p = std::make_unique<int>(42);
```

```
auto buffer = std::make_unique<Buffer>(1024);
```

```
// shared ownership
```

```
auto shared_buffer = std::make_shared<Buffer>(2048);
```

```
{
```

```
    auto another_ref = shared_buffer; // shared ownership
```

```
} // another_ref goes out of scope, buffer still alive
```

```
// when shared_buffer goes out of scope, buffer is deleted
```

C++ smart pointer types: `weak_ptr<T>`

- `std::weak_ptr<T>` **non-owning reference to a `shared_ptr` resource**
- **Solves the problem of cyclic references** with `shared_ptr` that cause memory leaks
- **Useful for safe observation**

```
{  
    std::weak_ptr<int> wp(sp1);  
    // weak reference to sp1 (does not affect ref count)  
}
```

```
struct Node {  
    std::shared_ptr<Node> parent; // owning reference  
    std::vector<std::shared_ptr<Node>> children;  
};
```

```
void leak_example() {  
    auto root = std::make_shared<Node>();  
    auto child = std::make_shared<Node>();
```

```
    root->children.push_back(child);  
    child->parent = root; // cycle: root ↔ child  
} // refcount never reaches zero → leak
```

```
struct NodeFixed {  
    std::weak_ptr<NodeFixed> parent; // non-owning reference  
    std::vector<std::shared_ptr<NodeFixed>> children;  
};
```

C++: Rule of Three, Rule of Five, and Rule of Zero

Advanced Programming Languages: C++

C++: Rule of Three, Rule of Five, and Rule of Zero

- They are **guidelines for resource-owning classes** in C++
- Help decide **which special member functions** you need to implement
- **Evolved with the language:**
 - **Rule of Three** → pre-C++11
 - **Rule of Five** → post-C++11 (move semantics)
 - **Rule of Zero** → modern best practice (RAII + smart pointers)

C++: Rule of Three (pre-C++11)

- If your **class manages a resource manually (e.g., raw pointer)**, you **likely need**:
 - Destructor ($\sim T$) to free the resource
 - Copy constructor ($T(\text{const } T\&)$)
 - Copy assignment ($T\& \text{operator}=(\text{const } T\&)$)
- Copy constructor and assignment must **deep copy** to avoid double delete

```
class Buffer {  
    char* data;  
    std::size_t size;  
public:  
    Buffer(std::size_t n) : data(new char[n]), size(n) {}  
  
    // Rule of Three  
    ~Buffer() { delete[] data; }  
    Buffer(const Buffer&);           // deep copy  
    Buffer& operator=(const Buffer&); // deep copy  
};
```

C++: Rule of Five (C++11) to handle move semantics

- If you implement any of the three of the Rule of Three, you probably also need:
 - **Move constructor** (T(T&&))
 - **Move assignment** (T& operator=(T&&))
- To allow **efficient ownership transfer**, and avoid unnecessary **deep copies**

```
class Buffer {  
    char* data;  
    std::size_t size;  
public:  
    Buffer(std::size_t n) : data(new char[n]), size(n) {}  
    // Rule of Three  
    ~Buffer() { delete[] data; }  
    Buffer(const Buffer&);           // deep copy  
    Buffer& operator=(const Buffer&); // deep copy  
  
    // Rule of Five  
    Buffer(Buffer&&) noexcept;      // move  
    Buffer& operator=(Buffer&&) noexcept;  
};
```

C++: Rule of Zero (C++11), the modern best practice

- **Don't manage resources manually**
- **Use RAII types** (smart pointers, containers)
- Then you need **no custom special members, use compiler-generated destructor, copy/move are correct** (and handled by the smart pointers / containers)

```
#include <memory>

class Buffer {
public:
    explicit Buffer(std::size_t n)
        : data(std::make_unique<char[]>(n)), size(n) {}

    // No destructor, no copy/move logic needed
private:
    std::unique_ptr<char[]> data;
    std::size_t size;
};
```